

10 Projects to Do for better resume:

https://docs.google.com/document/d/1sCyf5H_yFfLtVDqO8cLqynvf3VISJTR3PF5hMRBKfAg/edit?usp=sharing

Register for free Masterclass:

Friday 27 Feb, 5 PM

<https://forms.gle/sTNJLDxqV7JmMyFi8>



I've launched LLD Live Cohort 2.0 do check to enroll if you need structured guidance and mentoring

<https://livecohortbypradeep.com/>

Choose Your Program only Few Seats left

LLD Live Cohort

₹9,999 **₹4,999**
Early Bird offer

- ✓ OOP + SOLID Deep Dive
- ✓ All Design Patterns with code
- ✓ 15+ most important LLD Questions with Live Coding
- ✓ Mock Interview & complete support to crack the interviews
- ✓ 8-10 weeks with 2hrs live classes SAT-SUN

[Enroll in LLD](#)

Most Popular

Live LLD + DSA for working professionals

₹9,999 **₹6,999**
Early Bird offer

- ✓ Full DSA Cohort
- ✓ Full LLD Cohort
- ✓ Priority Doubt Support
- ✓ Mock Interviews
- ✓ Lifetime Recordings

[Enroll Bundle](#)

Live DSA Cohort

₹7,999 **₹3,999**
Early Bird offer

- ✓ Live Coding Sessions
- ✓ All patterns covered
- ✓ Doubt Clearing
- ✓ 75-100 Questions
- ✓ 10-12 weeks with 1 hr live class SAT-SUN

[Enroll in DSA](#)

Phase 1: OOP Foundations (Week 1)

Must-know concepts

- **Encapsulation (private state, public behavior)**
- **Abstraction vs Interface**
- **Inheritance (when NOT to use it)**
- **Polymorphism (runtime > compile-time)**
- **UML Diagrams**

Interview expectations

- **Correct class responsibility**
- **Proper access modifiers**
- **Clean constructors**

Practice mini-designs

- Design a **Printer**
- Design a **BankAccount**
- Design a **Car** with Engine abstraction

Phase 2: SOLID Principles (Week 2)

This is non-negotiable for SDE-2.

SRP – Single Responsibility

One class = one reason to change

OCP – Open Closed

Open for extension, closed for modification

LSP – Liskov Substitution

Child should be replaceable by parent

ISP – Interface Segregation

Many small interfaces > one fat interface

DIP – Dependency Inversion

Depend on abstractions, not concretes

Always explain what problem each principle solves.

Phase 3: Design Patterns (Week 3–4)

Focus on when & why, not memorization.

Creational

- **Singleton (thread-safe)**
- **Factory**
- **Abstract Factory**
- **Builder**

Structural

- **Adapter**
- **Decorator**
- **Facade**
- **Composite**

Behavioral (Very important)

- **Strategy**
- **Observer**
- **Command**

- **State**
- **Chain of Responsibility**

Most LLD problems reduce to Strategy + Factory + Composition

Phase 4: Concurrency in LLD (Week 5)

SDE-2 often includes thread-safety discussion.

Must-know

- **synchronized**
- **volatile**
- **Lock, ReentrantLock**
- **Executors**
- **Thread-safe Singleton**
- **Producer-Consumer**

Apply in designs

- **Thread-safe Cache**
- **Rate Limiter**
- **Parking Lot entry system**

Phase 5: Core LLD Interview Problems (Week 6–8)

These are absolute must-designs

Beginner

- **Design a Logger**
- **Design a File System**
- **Design a Vending Machine**

Intermediate (SDE-2 Sweet Spot)

1. **Parking Lot System**
2. **Elevator (Lift) System**
3. **LRU Cache**
4. **Rate Limiter**
5. **Online Chess Game**
6. **Splitwise (Expense Sharing System)**
7. **Movie Ticket Booking System**
8. **ATM System**
9. **Notification System**
10. **File System (Simplified)**
11. **Ride Booking System (Uber/Ola)**
12. **URL Shortener**
13. **Logger System**
14. **Vending Machine**
15. **Online Shopping Cart**
16. **Hotel Booking System**
17. **Snake and Ladder Game**
18. **Library Management System**
19. **Task Scheduler**
20. **Cache with Expiry (TTL Cache)**

System Design Roadmap[These are must rest is optional]

Foundations

- Client–Server model
- HTTP / REST basics
- Latency vs Throughput
- CAP theorem

Scalability Basics

- Vertical vs Horizontal scaling
- Stateless vs Stateful services
- Load balancing basics

Databases

- SQL vs NoSQL
- Indexing & sharding
- Replication & consistency
- Read vs write optimization

Caching

- Why caching is needed
- Cache-aside pattern
- Redis / Memcached basics

- Cache invalidation strategies

API Design

- RESTful API design
- Idempotency
- Pagination & filtering
- Versioning

Asynchronous Processing

- Message queues
- Pub–Sub model
- Event-driven architecture
- Background jobs

Consistency & Reliability

- Strong vs eventual consistency
- Idempotent consumers
- Retry, timeout, circuit breaker

Distributed Systems Concepts

- Leader election
- Distributed locking
- Heartbeats & failure detection

Security & Observability

- Authentication & authorization
- Rate limiting

- Logging, monitoring, alerting

Interview Questions

- URL Shortener
- Chat application
- Feed system
- Rate Limiter
- File storage system

15 HIGH-SIGNAL LLD INTERVIEW PROBLEMS (NON-CLICHÉ)

CONCURRENCY / MULTI-THREADING

Rate Limiter (Token Bucket / Sliding Window)

 Asked at: Google, Uber, Atlassian, Amazon
Tests:

- Thread safety
- Time-based logic
- Atomic operations
- Strategy Pattern

Variants: per-user, per-API, distributed

Thread-Safe In-Memory Cache (LRU + TTL)

 Asked at: Amazon, Flipkart, Microsoft
Tests:

- **ConcurrentHashMap**
 - **Locks vs lock-free**
 - **Eviction policies**
 - **Background cleanup thread**
-

Producer–Consumer Queue with Backpressure

📌 Asked at: Amazon, Netflix
Tests:

- **Condition variables**
 - **Blocking queues**
 - **Bounded buffers**
 - **Fairness**
-

Scheduled Task Executor (Mini Cron)

📌 Asked at: Google, Uber
Tests:

- **DelayQueue**
 - **Thread pools**
 - **Priority scheduling**
 - **Time abstraction**
-

Pub-Sub System (In-Memory)

📌 Asked at: LinkedIn, Uber
Tests:

- Observer pattern
 - Thread-safe fan-out
 - Message ordering
 - Async delivery
-

Thread-Safe Connection Pool

📌 Asked at: Oracle, Amazon
Tests:

- Object pooling
 - Semaphore
 - Timeout handling
 - Resource cleanup
-

STATE + CONCURRENCY (INTERESTING COMBOS)

Multiplayer Game Lobby System

📌 Asked at: Microsoft Gaming, Amazon
Tests:

- Concurrent joins
 - Matchmaking
 - State transitions
 - Lock granularity
-

Online Voting / Polling System

📌 Asked at: Meta, Atlassian

Tests:

- Atomic updates
 - Idempotency
 - Race conditions
 - Consistency guarantees
-

Ride-Matching Engine (Core Logic Only)

📌 Asked at: Uber, Ola

Tests:

- Concurrent updates
 - Strategy pattern
 - State machines
 - Fair matching
-

EVENT / ASYNC SYSTEMS

Notification Dispatcher (Email / SMS / Push)

📌 Asked at: Amazon, Google

Tests:

- Retry logic
- Async queues
- Thread pools
- Failure handling

Distributed ID Generator (Snowflake-like)

📌 Asked at: Twitter, LinkedIn
Tests:

- Thread safety
- Time ordering
- Bit manipulation
- Clock drift handling

Circuit Breaker

📌 Asked at: Netflix, Amazon
Tests:

- State pattern
- Concurrent counters
- Timeout windows
- Failure thresholds

DESIGN + REAL-WORLD CONCURRENCY

Log Aggregator (In-Memory)

Asked at: Splunk-like companies
Tests:

- Concurrent writes
- Buffering
- Batch flush

- Memory limits
-

Job Scheduler with Dependencies

Asked at: Airbnb, Google

Tests:

- DAG handling
 - Thread pools
 - Deadlock prevention
 - Dependency resolution
-

Chat System (Core Engine Only)

 Asked at: Meta, Amazon

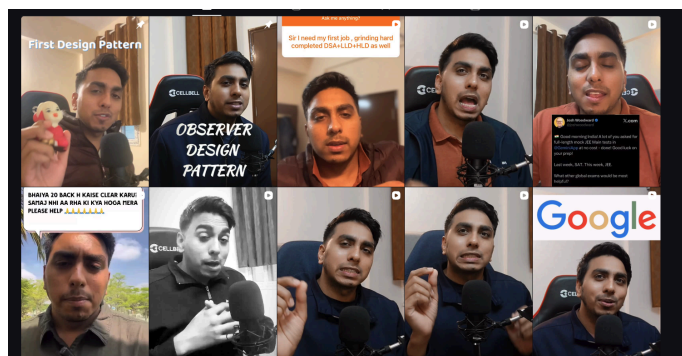
Tests:

- Message ordering
- Concurrent delivery
- Fan-out models
- Consistency tradeoffs



I've launched LLD Live Cohort 2.0 do check to enroll if you need structured guidance and mentoring:

<https://livecohortbypradeep.com/>





pradeep_kumar_iitd 

Pradeep kumar

426 posts 44.3K followers 1,964 following

Education

Senior SDE

 IIT Delhi | GATE-Ranker | Ex-Qualcomm

 crack System Design Interviews

 shorturl.at/job6Tp and 4 more